МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №1

з курсу: „ Обробка надвеликих масивів даних”

виконав студент 5го курсу
групи КН-51мп
Сидько Н.С.

КИЇВ
2025

Мета роботи

Ознайомитися з основними поняттями та можливостями Apache Spark та алгоритму MapReduce, отримати базові навички налаштування середовища PySpark для обробки великих даних.

Хід роботи

1) Встановлення JDK на комп'ютері.



Рис.1 JDK

2) Встановити Apache Spark на комп'ютері. Можна завантажити останню версію з офіційного сайту Apache Spark

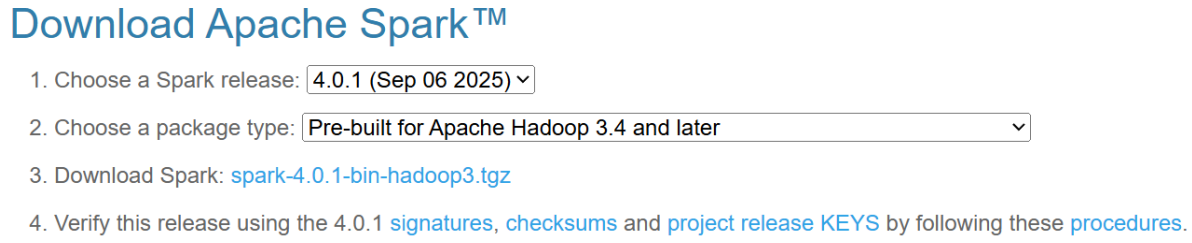


Рис.2 Apache Spark

3) Встановити Python на комп'ютері. PySpark підтримує Python версії 2.7 та 3.4 і вище.

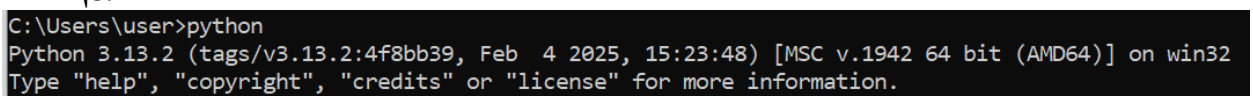


Рис.3 Python

4) Встановлення PySpark на комп'ютер
У cmd прописати pip install pyspark

5) Пропишемо шлях у PATH

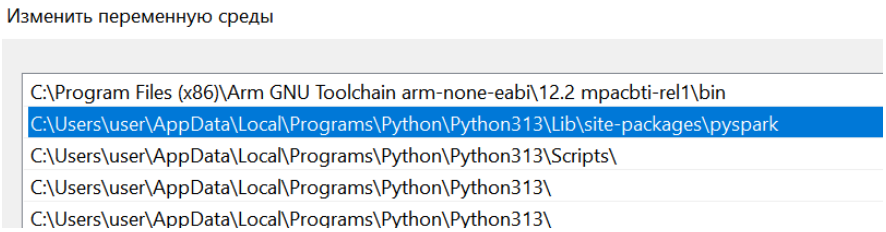


Рис.4 Path

6) Запуск PySpark

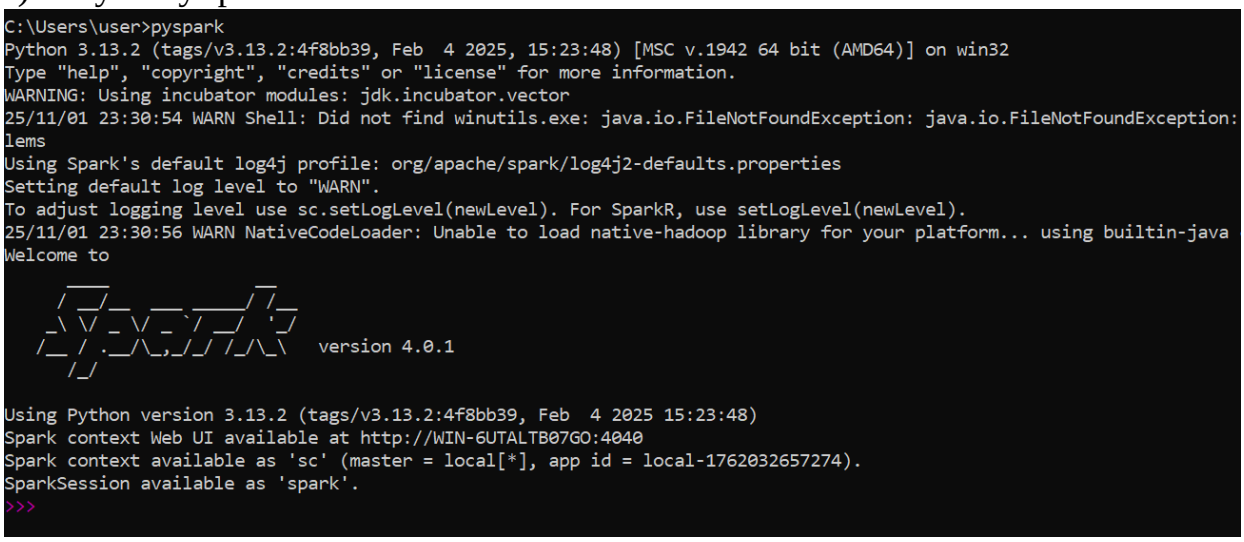


Рис.5 PySpark

Word Count

Задача Word Count полягає у визначенні, скільки разів кожне слово зустрічається у текстовому файлі. У контексті розподілених обчислень Spark ця задача зазвичай виконується через модель Map – Reduce, де етап Map відповідає за розбиття тексту на окремі слова, а Reduce за підсумовування кількості повторень кожного слова. Також додатковою вимогою було збереження порядку появи слів у тексті, тобто вивід результату мав відповідати послідовності слів у початковому файлі.

Для виконання роботи було створено два текстові файли A.txt та B.txt з нижче наведеним змістом:

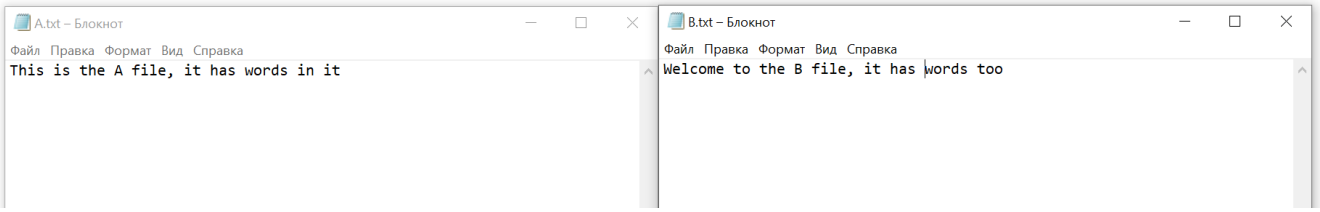


Рис.6 Текстові файли A.txt та B.txt

Далі була написана програма на основі PySpark з використанням RDD. На початку створюється SparkContext, що дозволяє працювати з розподіленими даними. Після цього файл зчитується за допомогою textFile(), і текст розбивається на слова за допомогою регулярного виразу, який підтримує як латиницю, так і українські символи. Важливо, що всі слова спершу переводяться у нижній регістр, що дозволяє уникнути подвійного підрахунку слів, які були записані з різними великими/малими літерами.

Особливістю моєї реалізації є використання операції zipWithIndex(), яка додає для кожного слова унікальний порядковий номер. Це дає можливість зберегти точну послідовність слів, взяту безпосередньо з тексту. Після цього RDD сортується за цими індексами, що дозволяє отримати список слів у вихідному порядку.

Сам код наведений у Додатку 1, а результат роботи коду наведений нижче:

```
A.txt
this - 1
is - 1
the - 1
a - 1
file - 1
it - 2
has - 1
words - 1
in - 1

B.txt
welcome - 1
to - 1
the - 1
b - 1
file - 1
it - 1
has - 1
words - 1
too - 1
```

Рис. 7 Результат роботи коду

Inverted Index

Задача Inverted Index полягає у побудові структури, яка для кожного унікального слова визначає список файлів, у яких це слово зустрічається. Такий індекс широко використовується в системах пошуку, оскільки дозволяє

швидко знаходити документи, що містять певні ключові слова. Тут потрібно було реалізувати побудову інвертованого індексу на основі PySpark з використанням моделі Map-Reduce, а також доповнити індекс позицією слова в документі offset (offset дає змогу визначити точне місце слова у байтовому представленні файлу)

Для виконання завдання було використано аналогічний до попереднього набір текстових файлів (A.txt, B.txt)

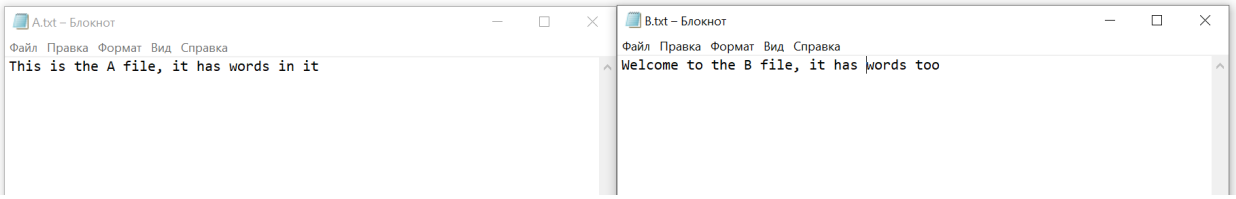


Рис. 8 Текстові файли

На початку виконання програми створюється SparkContext, що забезпечує можливість розподіленої обробки файлів. Після цього вміст кожного файлу зчитується у звичайному Python-режимі, а далі перетворюється в RDD через sc.parallelize().

Процес побудови індексу починається з етапу Map, який у моїй реалізації виконано через функцію map_document(). Ця функція проходить текст покроково, розбиває його на рядки, а кожне слово знаходить за допомогою регулярного виразу. Важливо, що під час розбору визначається не лише слово, але й його байтовий зсув (offset) у файлі. Він обчислюється шляхом накопичення кількості байтів у рядках та додаванням позиції слова в межах рядка. Кожне слово відображається у пару виду (word, "A.txt@offset123").

```
def map_document(filename, text):  1 usage
    pairs = []
    offset = 0

    for line in text.splitlines(keepends=True):
        line_clean = line.rstrip("\r\n").lower()

        for match in re.finditer( pattern: r"[A-Za-zA-Яа-яІіЇїЄєҐґ0-9]+", line_clean):
            word = match.group(0)
            start = offset + match.start()
            pairs.append((word, f"{filename}@offset{start}"))

        offset += len(line.encode("utf-8"))

    return pairs
```

Рис. 9 Функція map_document.

Далі виконується етап Reduce, який реалізовано у функції reduce_documents(). Вона отримує всі значення для певного слова (тобто всі вхождения у файлах), відфільтровує дублікати через множину set(), сортує результати та формує підсумковий рядок зі списком файлів. У результаті для кожного слова формується перелік файлів та позицій, де це слово зустрілося

```
def reduce_documents(file_list):  1 usage
    unique_files = sorted(set(file_list))
    return ", ".join(unique_files)
```

Рис. 10 Функція reduce_documents

Фінальна обробка виконується через типовий для RDD ланцюжок операцій:

```
inverted_index = (
    mapped_rdd.groupByKey()
    .mapValues(reduce_documents)
    .sortByKey()
    .collect()
)
```

Рис. 11 Inverted Index

groupByKey() збирає всі позиції слова разом, mapValues() застосовує редуктор, а sortByKey() сортує індекс за словами (повний код можна побачити у Додатку 2). Тоді маємо ось такий результат роботи коду :

```
Inverted Index
a           A.txt@offset12
b           B.txt@offset15
file        A.txt@offset14, B.txt@offset17
has         A.txt@offset23, B.txt@offset26
in          A.txt@offset33
is          A.txt@offset5
it          A.txt@offset20, A.txt@offset36, B.txt@offset23
the         A.txt@offset8, B.txt@offset11
this        A.txt@offset0
to          B.txt@offset8
too         B.txt@offset36
welcome     B.txt@offset0
words       A.txt@offset27, B.txt@offset30
```

Рис. 12 Результат роботи коду

Висновок.

У ході лабораторної роботи були реалізовані дві класичні задачі розподіленої обробки текстових даних: Word Count та Inverted Index (із використанням PySpark та MapReduce).

У першій задачі було продемонстровано механізм підрахунку частоти слів за допомогою розподілених трансформацій RDD, включно з токенізацією тексту, формуванням пар (слово, 1) та агрегуванням результатів.

У другій задачі було побудовано інвертований індекс, який встановлює відповідність між словами та файлами, у тому числі із використанням обчислення байтових offset-позицій, що розширює точність базової реалізації.

Обидві програми продемонстрували ефективність моделі MapReduce та гнучкість API PySpark для обробки текстових даних, а також сформували розуміння принципів трансформацій, дій, лінивої обробки, групування і сортування у RDD.

Теоретичні питання

1. Що таке великі дані (Big Data) та як вони відрізняються від звичайних даних? Які проблеми виникають при обробці великих даних&

Великі дані (Big Data) це такі обсяги / потоки даних, які є занадто великими, швидкими або різноманітними для обробки традиційними інструментами (напр. звичайні БД чи локальні програми). Їх характеризують трьома класичними властивостями: volume, velocity та variety.

На відміну від звичайних даних, такі дані потрібно обробляти паралельно на кластері машин, оскільки один комп’ютер не може швидко виконати всі необхідні операції. Проблеми, що виникають при роботі з Big Data включають зберігання великих обсягів даних, масштабування інфраструктури, обробку в реальному часі, забезпечення відмовостійкості та оптимізацію алгоритмів для роботи у розподіленому середовищі.

2. Як працює алгоритм MapReduce? Які етапи включає цей алгоритм та як вони пов'язані між собою?

Алгоритм MapReduce складається з двох основних етапів це власне Map і Reduce, які виконуються послідовно, але на великій кількості вузлів кластера. На етапі Map кожний вузол обробляє частину даних і перетворює їх на набір пар (ключ, значення). Наприклад для Word Count це пари (слово, 1). Після цього система автоматично виконує етап Shuffle (групує всі значення за однаковими ключами), на етапі Reduce кожен ключ обробляється окремим вузлом, а всі значення агрегуються. Етапи пов’язані тим, що Reduce не може початися, доки Map не створить проміжні ключі, а Shuffle не згрупує їх. Така модель дозволяє ефективно масштабувати обчислення на багато вузлів.

3. Що таке Apache Spark та як він пов'язаний з алгоритмом MapReduce? Які компоненти входять до складу Apache Spark та як вони взаємодіють між собою?

Apache Spark це високопродуктивний фреймворк для розподіленої обробки великих даних, який є логічним наступником MapReduce. На відміну

від Hadoop MapReduce, Spark виконує обчислення в оперативній пам’яті, що значно підвищує швидкість роботи, особливо для повторюваних задач.

Spark підтримує ту ж логіку Map - Reduce, але надає її у вигляді широкого набору трансформацій та дій над RDD, DataFrame та Dataset.

До складу Spark входять такі компоненти:

1. Spark Core — базова система обчислень та робота з RDD;
2. Spark SQL — модуль для роботи з табличними даними;
3. Spark Streaming — обробка потокових даних;
4. MLlib — бібліотека машинного навчання;
5. GraphX — робота з графами.

Ці модулі взаємодіють через єдине ядро Spark та виконуються в одному кластерному середовищі.

4. Як працює PySpark? Які основні поняття та технології використовуються в PySpark для обробки великих даних?

PySpark це Python інтерфейс до Apache Spark, який дозволяє використовувати можливості розподіленої обробки в Python-коді. Основою роботи PySpark є RDD (Resilient Distributed Dataset) (розподілений набір даних, що зберігається на різних вузлах кластера)

PySpark працює за принципом ледачих трансформацій: операції map, flatMap, filter не виконуються одразу, а лише будують DAG-план, який виконується після дії (collect(), count(), saveAsTextFile()). Також використовуються такі поняття, як трансформації, дії, партиції, виконавці (executors) та драйвер-програма. PySpark дозволяє працювати з RDD, DataFrame, SQL, потоками, що робить його універсальним у сфері Big Data.

5. Які є методи обробки даних в PySpark? Які операції можна виконувати з даними в PySpark та як вони впливають на швидкість обробки даних?

PySpark підтримує два основні типи операцій: трансформації та дії.

1)Трансформації включають map, flatMap, filter, reduceByKey, groupByKey, sortBy, join, distinct. Вони створюють нові RDD, але не виконуються негайно.

2)Дії (collect, take, saveAsTextFile, count) запускають виконання всього DAG-плану. На продуктивність найбільше впливають операції, що викликають Shuffle (наприклад, groupByKey або sortByKey), оскільки вони потребують пересилання даних між вузлами. Натомість операції типу map чи filter є локальними і виконуються швидко. Spark також оптимізує обробку через кешування (cache()) та розподіл даних між партиціями.

Додаток 1:

```
from pyspark import SparkConf, SparkContext
import os
import re
import warnings

warnings.filterwarnings("ignore")
```

```
def create_spark(app_name="WordCount-RDD"):
    conf = SparkConf().setAppName(app_name).setMaster("local[*]")
    sc = SparkContext(conf=conf)
    sc.setLogLevel("ERROR")
    return sc

def count_words_in_order(sc, file_path):
    text_rdd = sc.textFile(file_path)
    token_re = re.compile(r"[A-Za-zA-Яа-яІіЇїЄєҐґ0-9]+")

    words_with_index = (
        text_rdd
        .flatMap(lambda line: token_re.findall(line.lower()))
        .zipWithIndex()
        .map(lambda wc: (wc[1], wc[0]))  # (index, word)
    )

    words_list = [w for _, w in sorted(words_with_index.collect(), key=lambda x: x[0])]

    counts = {}
    for w in words_list:
        if w not in counts:
            counts[w] = 1
        else:
            counts[w] += 1

    ordered_results = [(w, counts[w]) for w in counts.keys()]
    return ordered_results


def print_result(filepath, results):
    filename = os.path.basename(filepath)
    print(f"\n{filename}")
    for word, count in results:
        print(f"{word} - {count}")


def main():
    sc = create_spark()
    files = [
        r"C:\Users\user\PycharmProjects\BD_Lab_1WC\data\A.txt",
        r"C:\Users\user\PycharmProjects\BD_Lab_1WC\data\B.txt"
    ]
    for f in files:
        result = count_words_in_order(sc, f)
        print_result(f, result)

    sc.stop()

if __name__ == "__main__":
    main()
```

Додаток 2:

```
from pyspark import SparkConf, SparkContext
import os, re, sys

def prepare_spark(app_name="InvertedIndexRDD", master="local[*]"):
    conf = (
        SparkConf()
        .setAppName(app_name)
```

```
        .setMaster(master)
        .set("spark.pyspark.python", sys.executable)
        .set("spark.pyspark.driver.python", sys.executable)
    )
    sc = SparkContext(conf=conf)
    sc.setLogLevel("ERROR")
    return sc

def tokenize_line(line):
    token_pattern = re.compile(r"[A-Za-zA-Яа-яЇїЇїЄєҐґ0-9]+")
    return token_pattern.findall(line.lower())

def map_document(filename, text):
    pairs = []
    offset = 0

    for line in text.splitlines(keepends=True):
        line_clean = line.rstrip("\r\n").lower()

        for match in re.finditer(r"[A-Za-zA-Яа-яЇїЇїЄєҐґ0-9]+", line_clean):
            word = match.group(0)
            start = offset + match.start()
            pairs.append((word, f"{filename}@offset{start}"))

        offset += len(line.encode("utf-8"))

    return pairs

def reduce_documents(file_list):
    unique_files = sorted(set(file_list))
    return ", ".join(unique_files)


def build_inverted_index(input_dir):
    sc = prepare_spark()
    all_files = [
        (fname, open(os.path.join(input_dir, fname), encoding="utf-8").read())
        for fname in os.listdir(input_dir)
        if fname.endswith(".txt")
    ]

    docs_rdd = sc.parallelize(all_files)
    mapped_rdd = docs_rdd.flatMap(lambda kv: map_document(kv[0], kv[1]))
    inverted_index = (
        mapped_rdd.groupByKey()
        .mapValues(reduce_documents)
        .sortByKey()
        .collect()
    )
    sc.stop()
    return inverted_index

def show_results(results):
    print("=== Inverted Index ===")
    for word, files in results:
        print(f"{word:<10} {files}")

if __name__ == "__main__":
    data_dir = r"C:\Users\user\PycharmProjects\BD_Lab_1IN\data"
```

```
    result = build_inverted_index(data_dir)
    show_results(result)
```